

Μη σχεσιακές βάσεις δεδομένων

Από τους πίνακες στα έγγραφα

Εβδομάδα 1

Μέρος 3ο



Αθανάσιος Χριστόπουλος, Ph.D.

Αν. Καθηγητής, Τμήμα Φυσικών Επιστημών. Κέντρο Μαθησιακής Αναλυτικής, Πανεπιστήμιο του Τούρκου, Φινλανδία
atchri@utu.fi

Τι περιέχει το Μέρος 3;

- 1 **Εγκατάσταση** της MongoDB και του mongosh στον υπολογιστή σας.
- 2 Οι έννοιες
βάση, συλλογή, έγγραφο, BSON και τύποι δεδομένων
- 3 Ο σχεδιασμός
ενσωμάτωση ή αναφορά
- 4 Οι πρώτες εντολές στο mongosh, και τι καλύπτουν από την **Εργασία 2**.

Κάθε εντολή εμφανίζεται με την έξοδό της. Στις διαφάνειες «**Δοκιμάστε**» την εκτελείτε κι εσείς.

Λογισμικό

**MongoDB Community
Server**

mongodb.com/try/download/community
Windows x64, πακέτο msi

mongosh

mongodb.com/try/download/shell
Windows x64 (10+), πακέτο msi

Είναι το **κέλυφος** όπου γράφουμε τις εντολές, και **δεν περιλαμβάνεται** στην εγκατάσταση του διακομιστή.

macOS ή Linux

[MongoDB Manual: Install MongoDB Community Edition](#), οδηγίες ανά σύστημα.

Παλαιότερα εγχειρίδια γράφουν mongo: το κέλυφος αυτό έχει **αντικατασταθεί από το mongosh**.

Εγκατάσταση

Στον υπολογιστή σας, με τα δύο αρχεία **msi**:

- 1 Εκτελέστε το **msi** του διακομιστή **Complete**, με το «**Install MongoDB as a Service**», χωρίς το «**Install MongoDB Compass**»
- 2 Εκτελέστε το **msi** του **mongosh**.
- 3 Ανοίξτε νέο τερματικό και γράψτε **mongosh**.
Η προτροπή **test>** σημαίνει επιτυχία. Οι προειδοποιήσεις πριν από αυτήν είναι **αναμενόμενες**.
- 4 Αν εμφανιστεί μήνυμα λάθους, γράψτε το στο **chat του Zoom**.

08:00

Όποιος δεν τελειώσει, συνεχίζει παράλληλα με το μάθημα. Οι διαφάνειες δείχνουν κάθε έξοδο.

Ο διακομιστής και το κέλυφος

9.0

η τρέχουσα έκδοση της MongoDB,
από τον Σεπτέμβριο του 2026

Ο διακομιστής τρέχει ως **υπηρεσία των Windows** και ξεκινά **μόνος του με τον υπολογιστή**.

- `mongosh`: **σύνδεση στον διακομιστή** από οποιοδήποτε τερματικό. `exit`: έξοδος.
- **Μήνυμα ECONNREFUSED**: ο διακομιστής δεν τρέχει. Υπηρεσίες των Windows → MongoDB Server → Έναρξη.
- **Χωρίς εγκατάσταση**: MongoDB Atlas, υπηρεσία στο νέφος με δωρεάν επίπεδο.

Βάση, συλλογή, έγγραφο

Βάση

Περιέκτης **συλλογών**. Ένας διακομιστής φιλοξενεί πολλές βάσεις.



Συλλογή

Ομάδα **εγγράφων** με παρόμοιο σκοπό, π.χ. οι χώρες.



Έγγραφο

Ζεύγη **πεδίου-τιμής**, π.χ. η Ελλάδα και ο πληθυσμός της.

Πλήρες όνομα συλλογής (namespace): βάση.συλλογή

`world.country`

Σήμερα: η βάση world και η συλλογή country, με χώρες της Ευρωπαϊκής Ένωσης.

Αντιστοιχία όρων με τις σχεσιακές βάσεις

Σχεσιακή βάση	MongoDB
Βάση δεδομένων	Βάση δεδομένων (database)
Πίνακας	Συλλογή (collection)
Γραμμή	Έγγραφο (document)
Στήλη	Πεδίο (field)
Πρωτεύον κλειδί	Το πεδίο <code>_id</code> , που δημιουργείται αυτόματα αν δεν το δώσουμε
Σύνδεση πινάκων (JOIN)	Εμφωλευμένο έγγραφο ή σύνδεση συλλογών με <code>\$lookup</code>

Η πλήρης αντιστοιχία, με τις εντολές: mongodb.com/docs/manual/reference/sql-comparison.

BSON και τύποι δεδομένων

Τύπος	Στο mongosh	Σημείωση
String	"Greece"	κείμενο UTF-8
Int32	42	ακέραιος που χωρά σε 32 bit
Double	8.5 , 30000000000	δεκαδικός, και οι μεγάλοι ακέραιοι
Int64	Long ("30000000000")	μεγάλος ακέραιος, ρητά
Decimal128	Decimal128 ("0.10")	ακριβείς δεκαδικοί, π.χ. ποσά
Date	new Date ("2025-11-22")	ημερομηνία και ώρα, σε UTC

Η MongoDB αποθηκεύει κάθε έγγραφο σε **BSON (Binary JSON)**: δυαδική μορφή, με τύπους που το JSON δεν έχει.

Το πεδίο `_id` και το `ObjectId`

12

bytes σε κάθε `ObjectId`: 4 για τη στιγμή δημιουργίας, 5 τυχαία, 3 μετρητής

Κάθε έγγραφο έχει **μοναδικό `_id`**. Αν δεν το δώσουμε, η MongoDB δημιουργεί ένα `ObjectId`.

- Το `_id` είναι το **πρωτεύον κλειδί** της συλλογής: μοναδικό και αμετάβλητο.
- Η **στιγμή δημιουργίας** διαβάζεται από το ίδιο το `ObjectId`: `.getTimestamp()`.
- Μπορούμε να δώσουμε **δικό μας `_id`**, αρκεί να είναι μοναδικό στη συλλογή.

Ενσωμάτωση ή αναφορά;

● Ενσωμάτωση

- Τα δεδομένα μπαίνουν **μέσα στο έγγραφο** (embedding) και διαβάζονται μαζί του.
- Σχέση **ένα-προς-λίγα**, π.χ. οι διευθύνσεις ενός φοιτητή.
- Τα δεδομένα δεν αλλάζουν ανεξάρτητα.

● Αναφορά

- Τα δεδομένα αλλάζουν ή διαβάζονται και μόνα τους.
- Σχέση **πολλά-προς-πολλά**, ή λίστα χωρίς όριο: ένα έγγραφο δεν ξεπερνά τα **16 MB**.
- Το έγγραφο κρατά **μόνο το _id** του άλλου εγγράφου (reference).

Η απόφαση παίρνεται **ανά σχέση**, όχι ανά βάση.

Μια σχεδίαση έχει συνήθως και ενσωμάτωση και αναφορά.

Από τους πίνακες στα έγγραφα

```
// συλλογή students
{ _id: 1042,
  name: "Μαρία Παπαδοπούλου",
  enrolments: [
    { course_id: "DB101", grade: 8.5 },
    { course_id: "LA201", grade: 9 } ] }

// συλλογή courses
{ _id: "DB101",
  title: "Βάσεις Δεδομένων" }
```

- Οι εγγραφές **ενσωματώνονται** στον φοιτητή: διαβάζονται πάντα μαζί του.
- Τα μαθήματα μένουν σε δική τους συλλογή, με **αναφορά** μέσω του `course_id`: αλλάζουν ανεξάρτητα.
- Αυτό ζητά η **Εργασία 2**: δύο τουλάχιστον πίνακες της Εργασίας 1, ο ένας εμφωλευμένος στον άλλον.

Πλοήγηση και νέα συλλογή

```
test> db
test
test> show dbs
admin      8.00 KiB
config    12.00 KiB
local      8.00 KiB
test> use world
switched to db world
world> db.createCollection("country")
{ ok: 1 }
world> show collections
country
```

- `db`: η **τρέχουσα βάση**, στην αρχή η `test`.
`show dbs`: οι βάσεις του διακομιστή, χωρίς ακόμη τη `world`.
- `use world`: αλλαγή στη βάση `world`. Η βάση δημιουργείται μαζί με την **πρώτη της συλλογή**.
- `createCollection`: νέα συλλογή. Χωρίς την εντολή, η συλλογή δημιουργείται με την **πρώτη εισαγωγή**.

Εισαγωγή ενός εγγράφου: insertOne

Εντολή

```
db.country.insertOne({
  name: "Greece",
  euSince: 1981,
  population: 10400000
})
```

Αποτέλεσμα

```
{
  acknowledged: true,
  insertedId: ObjectId('6ac03b88d126181d95e1c707')
}
```

- Στο mongosh τα ονόματα των πεδίων γράφονται και **χωρίς εισαγωγικά**: `name: "Greece"`.
- `acknowledged: true`: ο διακομιστής **επιβεβαίωσε την εγγραφή**.
- Το ObjectId στη δική σας οθόνη θα είναι **διαφορετικό**.

Δικό μας _id

Εντολή

```
db.country.insertOne({
  _id: 2,
  name: "Cyprus",
  population: 980000
})
```

Αποτέλεσμα

```
{ acknowledged: true, insertedId: 2 }
```

- Το _id το **δώσαμε εμείς**: η MongoDB δεν δημιουργεί ObjectId.
- Η Κύπρος δεν έχει πεδίο euSince: **ίδια συλλογή, άλλο σχήμα**.
- Δεύτερο έγγραφο με `_id: 2` **απορρίπτεται**, με σφάλμα διπλού κλειδιού.

Εισαγωγή πολλών εγγράφων: insertMany

Εντολή

```
db.country.insertMany([
  { name: "France", euSince: 1958, population: 68600000,
    capital: { name: "Paris", timezone: "Europe/Paris" } },
  { name: "Germany", euSince: 1958, population: 83600000,
    capital: { name: "Berlin", timezone: "Europe/Berlin" } }
])
```

Αποτέλεσμα

```
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6ac03b88d126181d95e1c708'),
    '1': ObjectId('6ac03b88d126181d95e1c709')
  }
}
```

- Μια **λίστα εγγράφων** σε μία εντολή.
- Τα δύο νέα έγγραφα έχουν **πεδίο capital**, που οι δύο πρώτες χώρες δεν έχουν.
- Ένα **_id** για **κάθε έγγραφο**, με τη σειρά εισαγωγής.
- Παλαιότερα εγχειρίδια γράφουν `insert()`: λειτουργεί, με **προειδοποίηση απόσυρσης**.

Επικύρωση σχήματος: \$jsonSchema

Εντολές

```
db.createCollection("cities", {
  validator: { $jsonSchema: {
    required: ["name", "country"],
    properties: { name: { bsonType: "string" } }
  } }
})
db.cities.insertOne({ name: "Athens" })
```

Αποτέλεσμα του insertOne, οι δύο πρώτες γραμμές

```
Uncaught:
MongoServerError: Document failed validation
```

- Κάθε πόλη **πρέπει να έχει** name και country, και το name να είναι κείμενο.
- Η Αθήνα χωρίς country **απορρίπτεται**. Στις επόμενες γραμμές της εξόδου, το missingProperties δείχνει ποιο πεδίο λείπει.
- **Χωρίς κανόνες**, όπως η συλλογή country, η MongoDB **δέχεται κάθε έγγραφο**.

Ανάγνωση: findOne και find

Εντολή

```
db.country.findOne()
```

Αποτέλεσμα

```
{
  _id: ObjectId('6ac03b88d126181d95e1c707'),
  name: 'Greece',
  euSince: 1981,
  population: 10400000
}
```

- `findOne()`: **ένα έγγραφο**, το πρώτο της συλλογής.
- `find()`: **όλα τα έγγραφα**.
Το mongosh τα δείχνει **ανά 20**, και με `it` εμφανίζονται τα επόμενα.
- Και οι δύο **δέχονται φίλτρο**.

Φίλτρο ισότητας

Εντολή

```
db.country.find({ name: "Greece" })
```

Αποτέλεσμα

```
[
  {
    _id: ObjectId('6ac03b88d126181d95e1c707'),
    name: 'Greece',
    euSince: 1981,
    population: 10400000
  }
]
```

- Το φίλτρο είναι **κι αυτό έγγραφο**:
{ πεδίο: τιμή }.
- Το mongosh δείχνει το αποτέλεσμα του find **σε αγκύλες, σαν λίστα**, ακόμη κι όταν ταιριάζει ένα μόνο έγγραφο.
- **Στην SQL:** WHERE name = 'Greece'.

Πλήθος: countDocuments

Εντολές

```
db.country.countDocuments()  
db.country.countDocuments({ euSince: 1958 })
```

Αποτελέσματα

```
4  
2
```

- **Χωρίς φίλτρο:** όλα τα έγγραφα.
- **Με φίλτρο:** όσα ταιριάζουν, εδώ οι δύο ιδρυτικές χώρες.
- Το παλαιό `count()` λειτουργεί με **προειδοποίηση απόσυρσης**.

Διαγραφή: `deleteOne` και `deleteMany`

Εντολή

```
db.country.deleteOne({ name: "Cyprus" })
```

Αποτέλεσμα

```
{ acknowledged: true, deletedCount: 1 }
```

- `deleteOne`: το **πρώτο έγγραφο** που ταιριάζει στο φίλτρο.
- `deleteMany`: **όλα** όσα ταιριάζουν.
- **Προσοχή:** το `deleteMany({})` **αδειάζει ολόκληρη τη συλλογή**, χωρίς επιβεβαίωση.

Η τελεία στα ερωτήματα

Εντολή

```
db.country.find({ "capital.name": "Paris" })
```

Αποτέλεσμα

```
[
  {
    _id: ObjectId('6ac03b88d126181d95e1c708'),
    name: 'France',
    euSince: 1958,
    population: 68600000,
    capital: { name: 'Paris', timezone: 'Europe/Paris' }
  }
]
```

- Με την τελεία το ερώτημα μπαίνει **μέσα στο εμφωλευμένο έγγραφο**.
- Το όνομα με την τελεία γράφεται **πάντα σε εισαγωγικά**.
- Ίδια σύνταξη και για **λίστες εγγράφων**:
"enrolments.course_id".

Η τελεία στο όνομα πεδίου

Εντολές

```
db.country.insertOne({ name: "Italy", "capital.name": "Rome" })
db.country.findOne({ name: "Italy" })
db.country.deleteOne({ name: "Italy" })
```

Αποτελέσματα του findOne και του deleteOne

```
{
  _id: ObjectId('6ac03b88d126181d95e1c70b'),
  name: 'Italy',
  'capital.name': 'Rome'
}
{ acknowledged: true, deletedCount: 1 }
```

- Με τελεία στο όνομα, η εισαγωγή δημιουργεί ένα πεδίο με όνομα «capital.name», **όχι εμφωλευμένο έγγραφο**.
- Το `find({ "capital.name": "Rome" })` **δεν επιστρέφει τίποτε**: στα ερωτήματα η τελεία **σημαίνει διαδρομή**.
- **Σωστά**: `capital: { name: "Rome" }`. Το `deleteOne` αφαιρεί το λανθασμένο έγγραφο.

● Ερώτημα

Τι επιστρέφει αυτό το ερώτημα;

Η Γαλλία έχει πεδίο capital με εμφωλευμένο έγγραφο: name «Paris» και timezone «Europe/Paris».

Τι επιστρέφει το ερώτημα που ακολουθεί;

```
db.country.find({ capital: "Paris" })
```

- A** Το έγγραφο της Γαλλίας
- B** Τίποτε
- Γ** Σφάλμα σύνταξης
- Δ** Όλα τα έγγραφα με πεδίο capital

01:00

Η απάντηση: τίποτε

Εντολή

```
db.country.find({ capital: "Paris" })
```

Τι εμφανίζεται; Μόνο η νέα προτροπή

```
world>
```

- Το mongosh **δεν τυπώνει τίποτε**: ούτε σφάλμα ούτε κενή λίστα.
- Το ερώτημα ζητά capital ίσο με το **κείμενο** "Paris", ενώ η τιμή είναι **έγγραφο**: καμία αντιστοιχία, κανένα σφάλμα.
- **Σωστά**: `{ "capital.name": "Paris" }`, με την τελεία.

Εργασία 2

● Σήμερα

- 1. Δημιουργία βάσης
- 2. Δημιουργία συλλογής
- 3. Δημιουργία εγγράφων
τουλάχιστον 4, τα 2 με εμφωλευμένα
- 12. Διαγραφή εγγράφου

● Εβδομάδα 2

- 4. Προβολή πεδίων, 5. Λογικοί τελεστές, 6. Τελεστές σύγκρισης
- 7. Αναζήτηση με regex, 8. Σύνθετα ερωτήματα
- 9. Ταξινόμηση και περιορισμός, 10. Συνάθροιση (pipeline), 11. Ενημέρωση
- Ευανάγνωστο JSON και μεταβλητές, σε κάποια ερωτήματα

Με όσα έγιναν σήμερα, οι ενότητες **1 έως 3 και 12 μπορούν να γίνουν ήδη, με τα δεδομένα της Εργασίας 1.**

Βασικά συμπεράσματα

- 1 Βάση, συλλογή, έγγραφο: η MongoDB αποθηκεύει **έγγραφα BSON**, καθένα με **μοναδικό _id**.
- 2 Η απόφαση παίρνεται ανά σχέση: **ενσωματώνεται** ό,τι ανήκει στο έγγραφο, και με **αναφορά** μπαίνει ό,τι αλλάζει ανεξάρτητα ή μεγαλώνει χωρίς όριο.
- 3 Στα ερωτήματα, **η τελεία** οδηγεί μέσα στο εμφωλευμένο έγγραφο. Στην εισαγωγή όμως δεν βάζουμε τελεία στο όνομα του πεδίου: γράφουμε έγγραφο μέσα σε έγγραφο.

Ερωτήσεις

Την επόμενη εβδομάδα: ερωτήματα στη συλλογή books, ομαδική άσκηση σε αίθουσες του Zoom, ενημέρωση και συνάθροιση.